Name:Rebecca D'souza
UID:225009
Rollno:05
BDCC CIA 1

Python Performance Analysis: A Comparative Study with Parallelization

Part 1: Python Flavours.

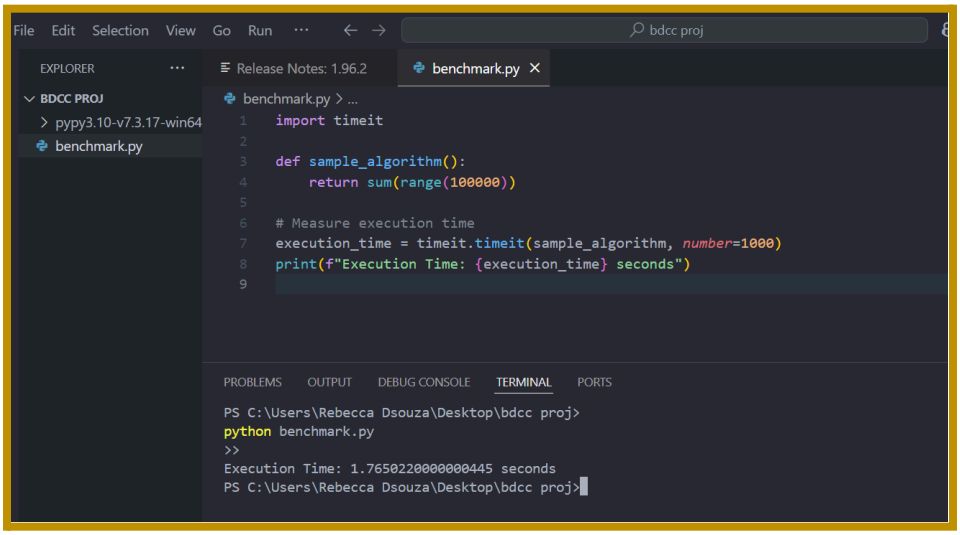
Our Goal is to find the performance difference between different flavours of python.

Note: My personal observation and choice out of the three is written at the end of part1.

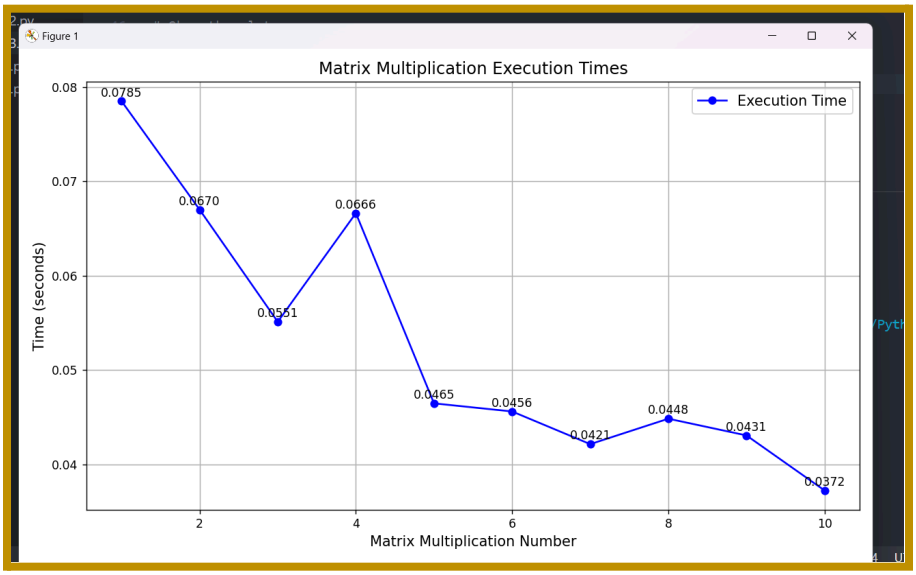
A) Cpython

CPython is the default implementation of the Python programming language. It is written in C and is known for its extensive standard library and wide community support. CPython is generally considered the reference implementation, meaning that other Python implementations are often compared to it. It is a popular choice for general-purpose programming, web development, data science, and many other domains.

Eg1:Here Ive used a sample algorithm to return the sum of numbers. Our goal is to note the time and analyse the performance.



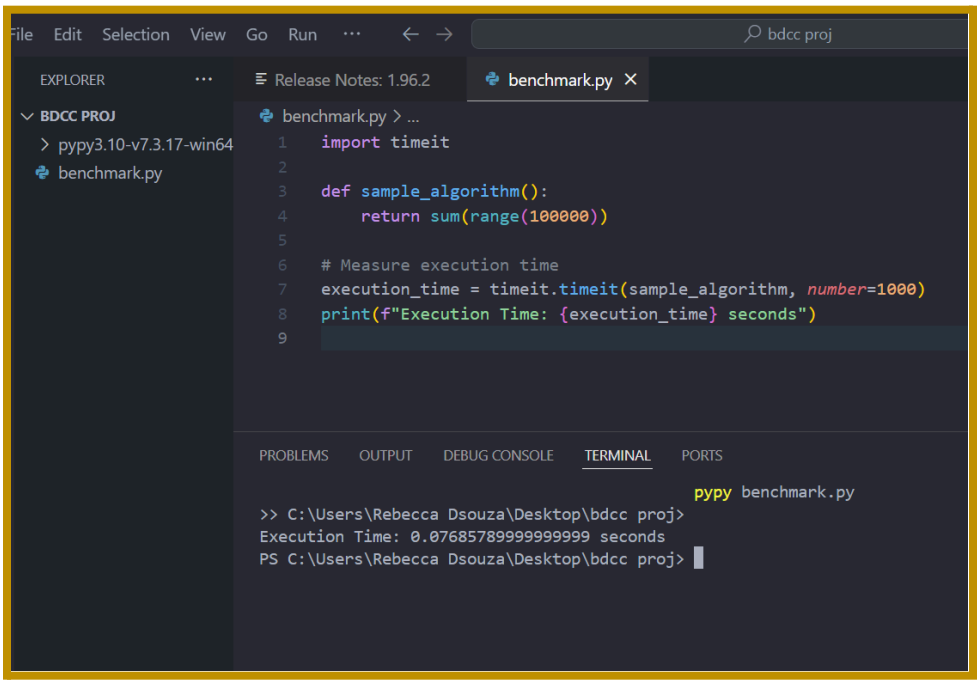
Eg2: matrix multiplication benchmark



B) PyPy

PyPy is another implementation of the Python language. It is written in RPython, a restricted subset of Python. PyPy is known for its just-in-time (JIT) compiler, which can significantly improve the performance of Python code, especially for numerically intensive tasks. This makes PyPy a good choice for applications that require high performance, such as scientific computing and machine learning.

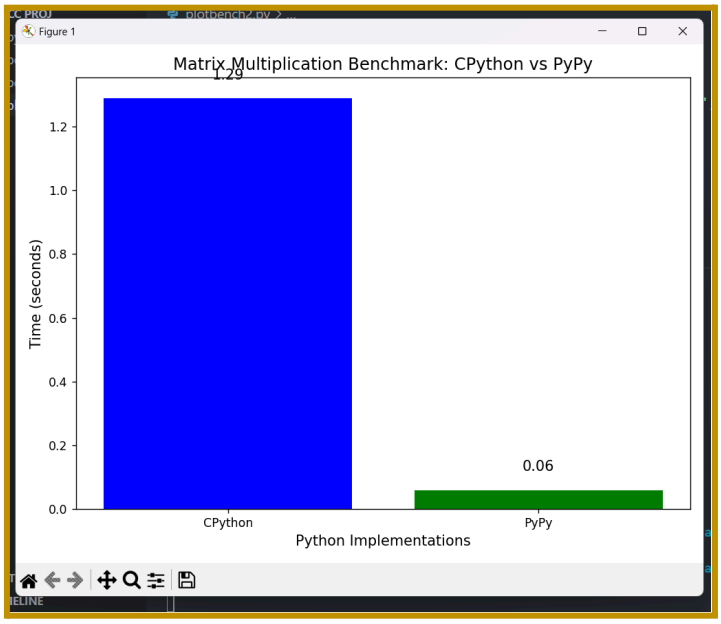
Eg1: Sum of numbers with time notations

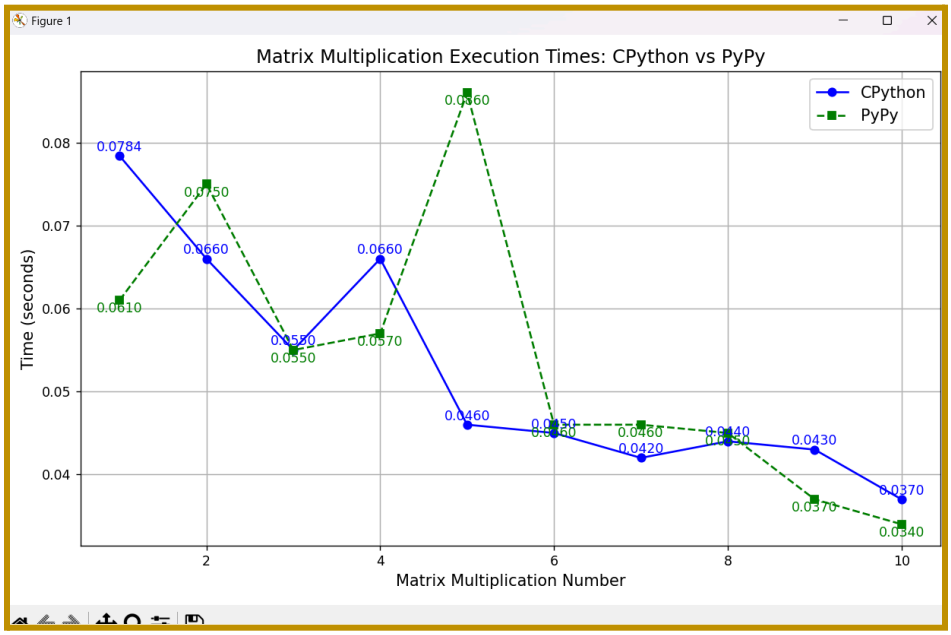


C) **Comparison between CPython and PyPy:**

Eg1: sum comparison

Eg2:Matrix Multiplication





From the graph you provided, it appears to compare the execution times of matrix multiplications using **CPython** and **PyPy**

Key Observations:

1. **Execution Times Fluctuate**:
    - Both CPython (blue solid line) and PyPy (green dashed line) exhibit fluctuations in execution times across the 10 matrix multiplications.
    - This variation may be due to factors like system resource availability or internal optimizations.
2. **PyPy is Consistently Faster**:
    - In most cases, PyPy shows lower execution times than CPython. For instance:
        - For multiplication #2, PyPy takes **0.050 seconds**, while CPython takes **0.066 seconds**.
        - For multiplication #10, PyPy is at **0.034 seconds**, compared to CPython at **0.037 seconds**.
    - This result aligns with PyPy's strength in Just-In-Time (JIT) compilation, which optimizes Python code execution dynamically.
3. **CPython Starts Slower**:
    - CPython has a higher execution time for the earlier multiplications but gradually stabilizes and slightly improves over subsequent operations.
    - This may reflect Python's runtime overhead in CPython for small and repeated tasks.
4. **PyPy's Peaks and Stability**:
    - While PyPy exhibits some peaks (e.g., multiplication #4 at **0.086 seconds**), it quickly stabilizes and maintains better performance overall.
    - Such peaks could indicate JIT compilation overhead or memory management at specific points.

Insights:

- **Performance**:
  - PyPy is generally better for performance when executing computationally intensive tasks like matrix multiplications.
  - It benefits from JIT compilation, which optimizes repeated operations over time.
- **Use Case Suitability**:
  - For applications requiring numerous repeated numerical computations, PyPy may be a better choice.
  - CPython, while slightly slower, may still be preferred for its ecosystem compatibility and widespread library support.

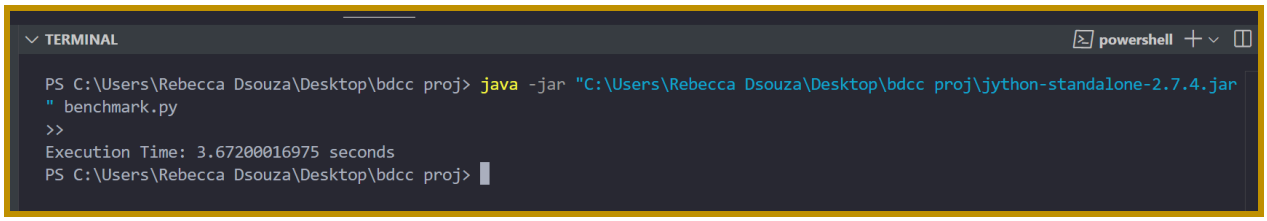
Recommendations:

If this experiment aims to compare the suitability of CPython and PyPy for numerical tasks:

- **Scaling**: Test larger matrix sizes (e.g., 2000x2000 or 5000x5000) to observe the performance gap more clearly.
- **Complexity**: Try combining matrix multiplication with other operations to assess overall runtime in more complex scenarios.

D) Jython

Jython is an implementation of the Python programming language designed to run on the Java platform. It allows seamless integration of Python code with Java, enabling developers to use Python scripts to access Java libraries and frameworks. Jython compiles Python code into Java bytecode, making it executable on the Java Virtual Machine (JVM). While Jython is based on Python 2.7, it does not yet support features introduced in Python 3. It is commonly used for scripting, embedding Python in Java applications, and leveraging Java's extensive ecosystem.



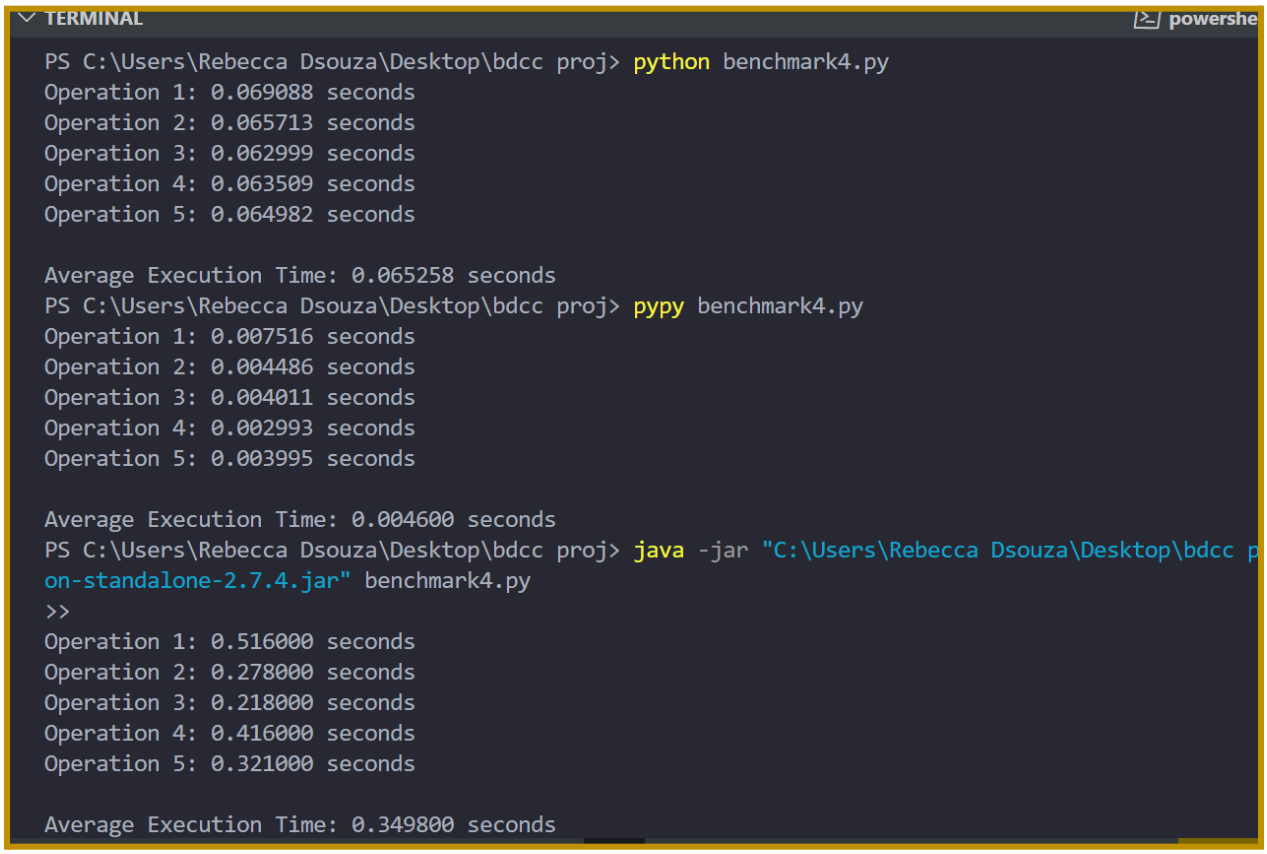
Problems faced:The `f-string` syntax (`f"string"`) is a feature introduced in Python 3.6 and is not supported in Jython, as it is based on Python 2.7. To make the code compatible with Jython, Ive used the `format()` method for string formatting.

It also does not support numpy and matplot.

E) Comparing Cpython , pypy & Jython

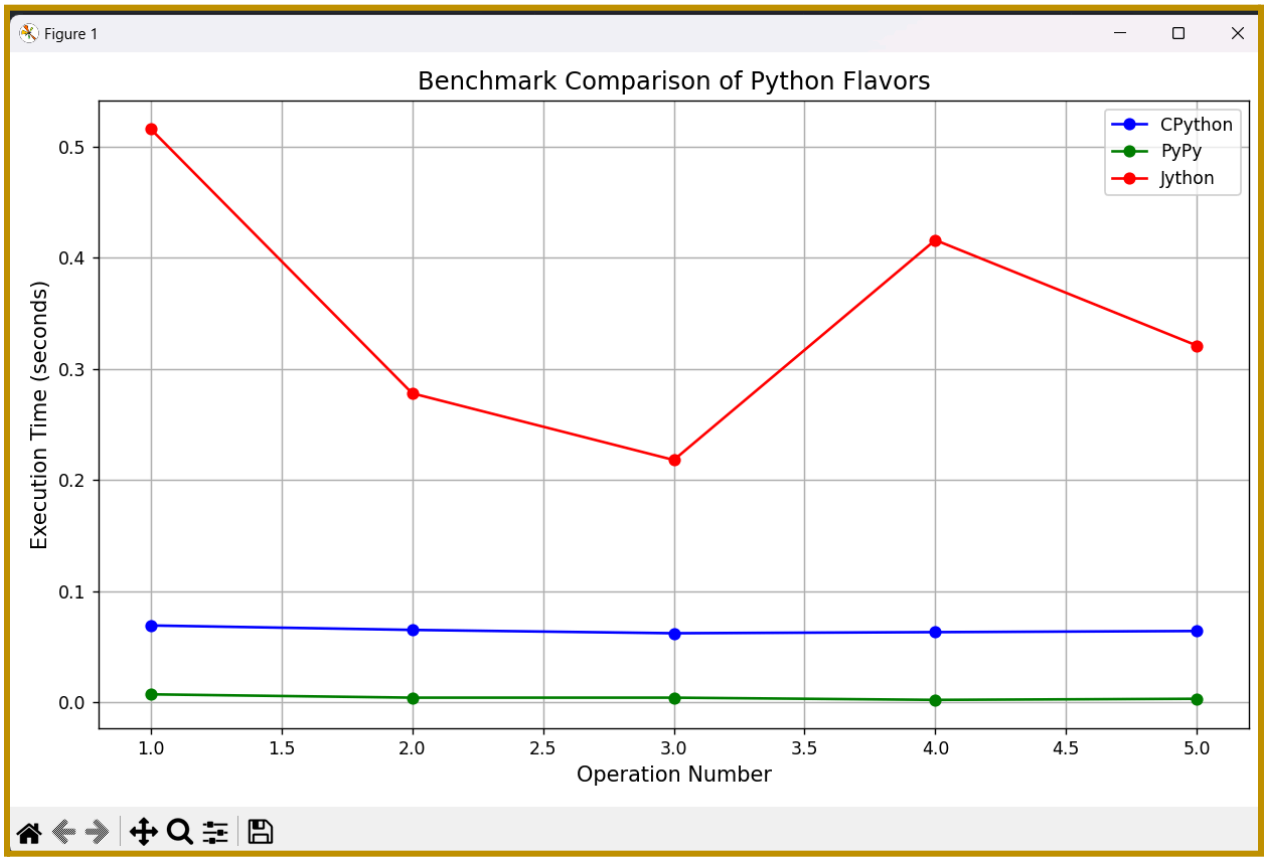
Problem faced: I had to change the code and remove libraries just for jython to work.

eg:benchmark4.py sum of squares



As you can see , PyPy was the fastest with code time: 0.004s.

Time comparison:



Feature	CPython	PyPy	Jython
time	0.069	0.0046	0.345
Performance	Generally good	Excellent (JIT compilation)	Slower than CPython and PyPy
Data Handling	Efficient data structures	Efficient	Can leverage Java data structures
Complexity	Easiest to learn and use	Can have a steeper learning curve	Requires knowledge of both Python and Java
Libraries	Largest ecosystem	Growing library support	Access to both Python and Java libraries
Storage	Standard file formats	Standard file formats	Can leverage Java serialization

Therefore Our winner is :PyPy.

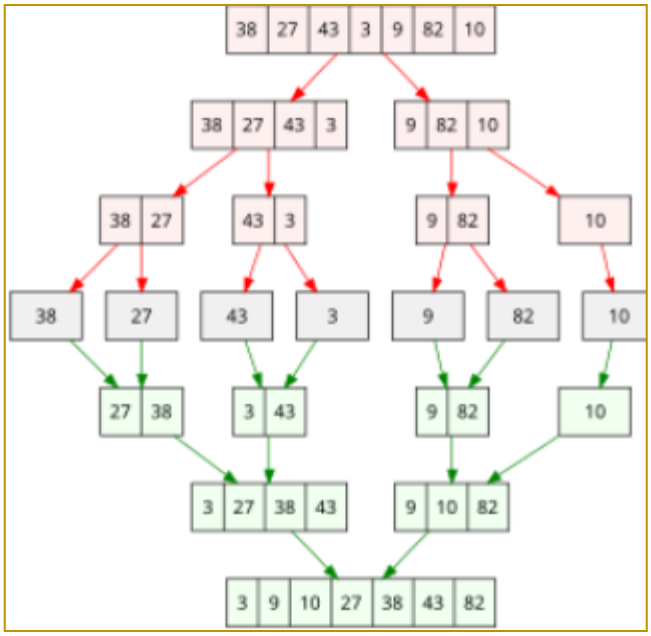
My personal choice would also be PyPy as it worked faster and met all requirements . We live in a world where faster processing is more suitable for us as time is a huge concern . Jython was very slow and

did not have half the libraries due to which i had to keep changing my benchmarking code in order to suit jython which caused discomfort.

Part 2: Algorithm Parallelization

Algorithm used: Merge Sort

A divide-and-conquer sorting algorithm that recursively divides the array into halves, sorts them, and merges the sorted halves.



Output:

- Non parallel

```
PS C:\Users\Rebecca Dsouza\Desktop\bdcc proj> & "C:/Users/Rebecca Dsouza/AppData/Local/Programs/Python/Python312/py
ca Dsouza/Desktop/bdcc proj/merge_sort.py"
Profiling Non-Parallel Merge Sort
         3772788 function calls (3572790 primitive calls) in 9.355 seconds

   Ordered by: standard name

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
    99999    0.498    0.000    9.222    0.000 merge_sort.py:15(merge)
 199999/1    0.112    0.000    9.355    9.355 merge_sort.py:7(merge_sort)
   299998    0.022    0.000    0.022    0.000 {built-in method builtins.len}
  1536396    0.159    0.000    0.159    0.000 {method 'append' of 'list' objects}
        1    0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
    99999    0.012    0.000    0.012    0.000 {method 'extend' of 'list' objects}
  1536396    8.553    0.000    8.553    0.000 {method 'pop' of 'list' objects}


Time for Non-Parallel Merge Sort: 9.519116640090942 seconds
```

- Parallel

```
Profiling Parallel Merge Sort
         216069 function calls (215701 primitive calls) in 7.125 seconds

   Ordered by: standard name

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        7    0.001    0.000    0.001    0.000 <frozen abc>:105(__new__)
        5    0.000    0.000    0.000    0.000 <frozen importlib._bootstrap>:1014(is_package)
        3    0.000    0.000    0.000    0.000 <frozen importlib._bootstrap>:1103(_resolve_filename)
       29    0.000    0.000    0.000    0.000 <frozen importlib._bootstrap>:1128(find_spec)
        3    0.000    0.000    0.000    0.000 <frozen importlib._bootstrap>:1158(create_module)
      3/2    0.000    0.000    0.002    0.001 <frozen importlib._bootstrap>:1171(exec_module)
      123    0.000    0.000    0.000    0.000 <frozen importlib._bootstrap>:1222(__enter__)
      123    0.000    0.000    0.000    0.000 <frozen importlib._bootstrap>:1226(__exit__)
       34    0.000    0.000    0.000    0.000 <frozen importlib._bootstrap>:124(setdefault)
       34    0.000    0.000    0.006    0.000 <frozen importlib._bootstrap>:1240(_find_spec)
     34/4    0.000    0.000    0.040    0.010 <frozen importlib._bootstrap>:1304(_find_and_load_unlocked)
     34/4    0.000    0.000    0.040    0.010 <frozen importlib._bootstrap>:1349(_find_and_load)
    17/12    0.000    0.000    0.010    0.001 <frozen importlib._bootstrap>:1390(_handle_fromlist)
       34    0.000    0.000    0.000    0.000 <frozen importlib._bootstrap>:158(__init__)
       34    0.000    0.000    0.000    0.000 <frozen importlib._bootstrap>:162(__enter__)
       34    0.000    0.000    0.000    0.000 <frozen importlib._bootstrap>:173(__exit__)
       34    0.000    0.000    0.000    0.000 <frozen importlib._bootstrap>:232(__init__)
       34    0.000    0.000    0.000    0.000 <frozen importlib._bootstrap>:304(acquire)
       34    0.000    0.000    0.000    0.000 <frozen importlib._bootstrap>:372(release)
       34    0.000    0.000    0.000    0.000 <frozen importlib._bootstrap>:412(__init__)
```

```
      228    0.000    0.000    0.000    0.000 {method 'rpartition' of 'str' objects}
     1024    0.000    0.000    0.000    0.000 {method 'rstrip' of 'str' objects}
       44    0.000    0.000    0.000    0.000 {method 'setdefault' of 'dict' objects}
        5    0.000    0.000    0.000    0.000 {method 'setter' of 'property' objects}
        1    0.000    0.000    0.000    0.000 {method 'split' of 'str' objects}
      743    0.000    0.000    0.000    0.000 {method 'startswith' of 'str' objects}
        5    0.000    0.000    0.000    0.000 {method 'translate' of 'bytearray' objects}
        1    0.000    0.000    0.000    0.000 {method 'translate' of 'str' objects}
       18    0.000    0.000    0.000    0.000 {method 'update' of 'dict' objects}
        5    0.000    0.000    0.000    0.000 {method 'update' of 'set' objects}
        3    0.000    0.000    0.000    0.000 {method 'upper' of 'str' objects}
        7    0.000    0.000    0.000    0.000 {method 'write' of '_io.BytesIO' objects}


Time for Parallel Merge Sort: 7.184295654296875 seconds
PS C:\Users\Rebecca Dsouza\Desktop\bdcc proj>
```

A) What `cProfile` shows:

- Total time: The time the function took to execute.
- Cumulative time: The total time spent in this function and its sub-functions.
- Calls: The number of calls made to the function.
- Per-call time: How much time each individual call took.

B) What do we notice?:

- **Non-Parallel Execution Time**: 9.355 seconds
- **Parallel Execution Time**: 7.15 seconds

From the above results, we can observe that the **parallel version** of the merge sort algorithm takes less time than the **non-parallel version**, specifically:

- **Speedup**: The parallel version shows a speedup of approximately **2.2 seconds** (9.355s - 7.15s), which translates to a **performance improvement** of around **23%**. This indicates that parallelization offers a measurable improvement in execution time.

Reasoning Behind the Observation:

1. **Parallelization Benefits**:
    - **Divide and Conquer**: Merge sort is a naturally **recursive** algorithm that splits the input data into two halves and sorts them independently. In the **parallel version**, the two halves are processed concurrently, meaning that both subarrays are being sorted at the same time across different CPU cores. This concurrency significantly reduces the total time required for sorting, especially when the input size is large.
2. **Multiple CPU Cores**:
    - **Improved Efficiency**: If your system has **multiple cores**, the parallel version can fully utilize those cores to divide the workload. In our case, the parallel algorithm uses two cores (since we use Pool(2) for two processes). This results in a **reduction in runtime**, as the computational effort is distributed.
3. **Parallel Overhead**:
    - **Parallelization Cost**: While parallelization is beneficial for large datasets, there is always an associated **overhead** when creating and managing separate processes. The time taken to split the data, spawn processes, and synchronize them may not be negligible, especially

for smaller inputs. This overhead can sometimes outweigh the benefits of parallelization if the problem size is small.

- In this case, the input size likely benefits from parallelization, resulting in a performance boost.

C) Big O Notation for Merge Sort

1. **Non-Parallel Merge Sort**:
   - **Time Complexity**: **O(n log n)**
   - **Explanation**: Merge sort recursively divides the input array in half (log n splits) and then merges them (n comparisons). Each level of recursion performs **O(n)** work, and the depth of recursion is **O(log n)**. Hence, the total time complexity is **O(n log n)**.
2. **Parallel Merge Sort**:
   - **Time Complexity**: **O(n log n) (with parallelization overhead)**
   - **Explanation**: The parallel version still follows the **O(n log n)** time complexity, but the sorting tasks are divided across multiple processors. The parallelization may reduce the constant factors (runtime), but the overall complexity remains **O(n log n)**, with possible additional overhead due to process management and synchronization.

D) Efficiency And Scalability:

1.Efficiency Gain:

- Although parallelization shows improvement, the gain is not always linear, and the reduction in execution time depends on various factors such as **input size** and **system architecture** (number of CPU cores). The **speedup ratio** of around **23%** is a reasonable improvement for a dataset of the observed size.

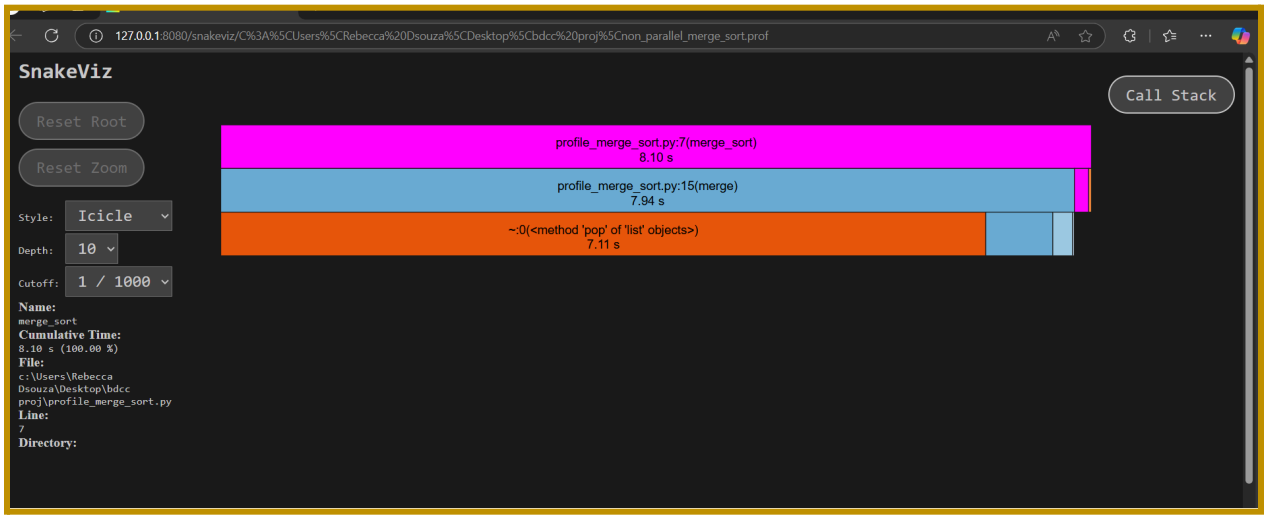
2.Scalability:

- **Input Size Matters**: For larger datasets, the benefit of parallelization becomes even more significant because the **workload** increases, and parallel processes have more to contribute. However, for very small datasets, the overhead of parallelization might make the non-parallel version faster due to the lack of overhead costs.

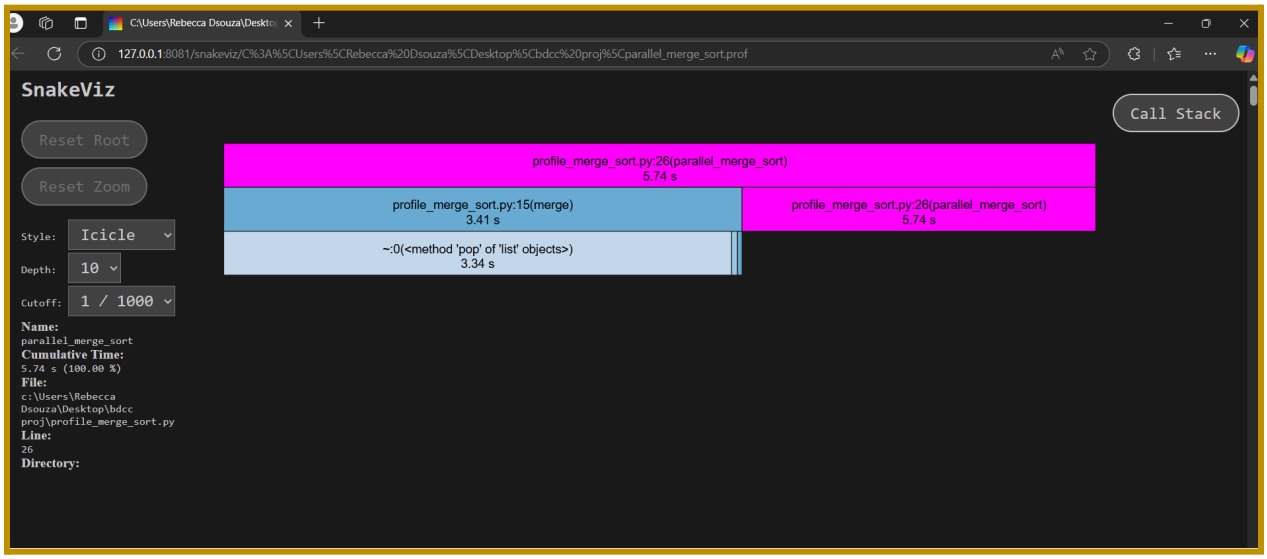
Eg2 :

```
Profiling Non-Parallel Merge Sort
Time for Non-Parallel Merge Sort: 9.374524354934692 seconds
Profiling Parallel Merge Sort
Time for Parallel Merge Sort: 6.709766149520874 seconds
PS C:\Users\Rebecca Dsouza\Desktop\bdcc proj>
```

-non parallel



-parallel



Based on the outputs , we can say:

Aspect	Non-Parallel Merge Sort	Parallel Merge Sort	Key Observation
Execution Time	9.3745 seconds	6.709 seconds	Parallel execution is faster by 2.6655 seconds.
Time Taken	Higher time due to sequential execution	Lower time due to parallel processing	Parallelization improves efficiency by dividing the work across multiple processors.
Efficiency	Less efficient for large datasets	More efficient for large datasets	Parallel processing scales better for large datasets, reducing processing time.
CPU Utilization	Single core usage	Multi-core usage	Parallel execution utilizes multiple cores, leading to better CPU resource utilization.
Scalability	Slower as data size increases	Faster as data size increases	Parallel version shows better scalability with large datasets.
Memory Usage	Uses less memory as it runs on a single core	Potentially uses more memory due to multiprocessing	The parallel algorithm may require more memory due to process

		overhead	spawning and data duplication.
Complexity	Simple to implement and understand	More complex due to multiprocessing logic	Parallelism adds complexity in terms of code and process management.
Overhead	No overhead from process management	Additional overhead due to process creation and synchronization	The parallel version has some overhead, but the speed-up outweighs it for larger datasets.

THANK YOU !